Software Architecture and Design Specification

Project: Smart Complaint Management System (SCMS)

Version: 1.0

Team Members: Rayudu Meghana [PES2UG24CS398], Rehaan Jose Mathew [PES2UG24CS400], Rakshita Biradar [PES2UG24CS397], Reet Anand [PES2UG24CS399]

Date: 14-10-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	14-10-2026		Initial architecture and design specification derived from SRS v1.0 (03-09-2026).

Approvals

Role	Name	Signature/Date
Team Lead		
Course Instructor		
QA / Test Owner		

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Smart Complaint Management System (SCMS), a web application for lodging, tracking and resolving citizen complaints. It translates the requirements in the SRS (v1.0) into components, interfaces, data structures and security controls that the team will implement in Sprint-1 and Sprint-2.

1.2 Scope

Covers SCMS services for three roles: Citizens (register, lodge complaints with evidence, track status, give feedback), Department Staff (handle assigned complaints, update status, record resolution) and Admins (manage users, categories, SLA rules, reassignment, reports and audit log). It covers all 15 functional, 5 non-functional and 5 security requirements of the SRS.

1.3 Audience

Developers, QA engineers, security reviewers, course instructors and future maintenance teams.

1.4 Definitions

Term	Meaning
SCMS	Smart Complaint Management System
SLA	Service Level Agreement – target time for resolving a complaint category
RBAC	Role-Based Access Control
TLS	Transport Layer Security (HTTPS)
OTP	One-Time Password sent by email/SMS for verification
ADR	Architecture Decision Record
STRIDE	Threat model: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
WCAG	Web Content Accessibility Guidelines
REST	Representational State Transfer API style
ORM	Object-Relational Mapper

2. Document Overview

2.1 How to use this document

Section 3 describes the architecture (components, pattern, technology, risks, traceability, security). Section 4 describes the detailed design (sequence diagrams, data model, workflow, API, error handling, UX). Developers should use Section 3.8 to find the module that owns each requirement and Section 4.3 for the interface contracts.

2.2 Related Documents

- Software Requirements Specification (SRS) v1.0 – SCMS-F-001..015, SCMS-NF-001..005, SCMS-SR-001..005
- Software Test Plan (STP) – test cases TC-Auth-01 … TC-Sec-05
- Requirements Traceability Matrix (RTM) – SRS Section 8
- Mini-Project Deliverables Part-2 – backlog and sprint plan

3. Architecture

3.1 Goals & Constraints

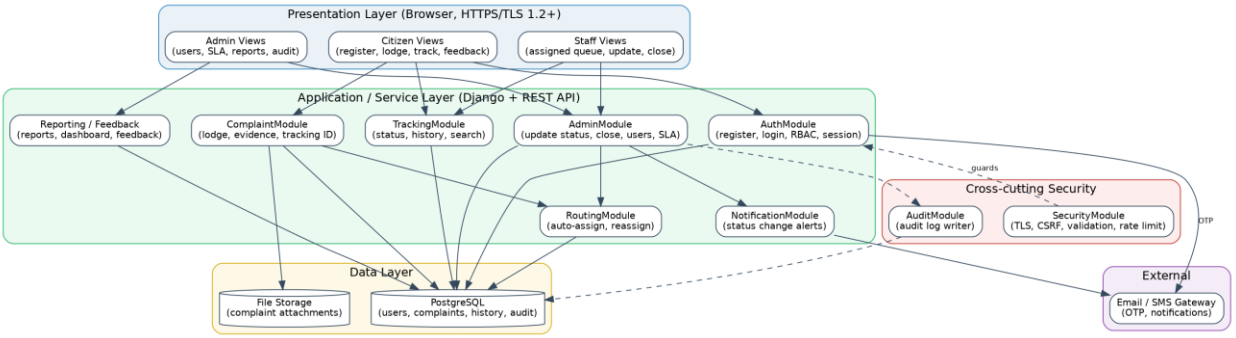
Goals: secure handling of citizen data; reliable (≥ 99.5% monthly availability); responsive (95th percentile ≤ 3 s); accessible (WCAG 2.1 AA); scalable to 10,000 citizens and 1,000 complaints/day; auditable (all admin/staff actions logged, records retained 5 years).

Constraints: web application for modern desktop/mobile browsers; Python (Flask/Django) or PHP backend and MySQL/PostgreSQL per SRS 2.3; HTTPS with TLS 1.2+; two one-week sprints (19–30 Oct 2026) for implementation; small student team; optional email/SMS provider (free-tier or console/mock in development).

3.2 Stakeholders & Concerns

Stakeholder	Concerns addressed by the architecture
Citizens	Easy lodging and tracking, privacy of personal data, timely notifications, accessible UI
Department Staff	Clear queue of assigned complaints, simple status update and closure workflow
Admins / Operations	Correct auto-routing, reassignment, SLA rules, reports, audit trail, availability
Security reviewers / Regulators	TLS, password hashing, RBAC on server, session security, audit logs, 5-year retention
Developers / Maintainers	Modularity, clear module boundaries, testability, simple deployment
Instructors / QA	Traceability from requirement to module to test case; working demo

3.3 Component (UML) Diagram



3.4 Component Descriptions

Component	Responsibility	Requirements
Web UI (Citizen / Staff / Admin views)	Responsive HTML/CSS/JS pages with role-specific dashboards and forms; calls the server over HTTPS.	UI 3.1, NF-001, NF-004
AuthModule	Registration with OTP verification, login/logout, password hashing, session management, role checks (RBAC).	F-001, F-002, F-003, SR-002, SR-003, SR-004
ComplaintModule	Complaint lodging, validation, evidence upload, unique tracking ID generation.	F-004, F-005, F-006
TrackingModule	Status lookup by tracking ID, chronological status history, search and filter.	F-007, F-008, F-009
RoutingModule	Automatic department assignment by category; admin reassignment to department/staff.	F-010, F-011
AdminModule	Status updates, resolution remarks, closure rules, user/category/SLA management.	F-012, F-013
NotificationModule	Generates and sends email/SMS on status change; records delivery result.	F-014
Reporting / Feedback	Complaint reports, dashboard analytics, citizen feedback after closure.	F-015
SecurityModule	TLS enforcement, CSRF protection, input validation, upload checks, rate limiting.	SR-001, SR-003
AuditModule	Writes immutable audit entries (timestamp, user ID, action details) for staff/admin actions.	SR-005

Component	Responsibility	Requirements
PostgreSQL database	Users, departments, categories, complaints, status history, notifications, feedback, audit log; archive for 5-year retention.	NF-003, NF-005
File storage	Stores validated complaint attachments (images/PDF) outside the web root; DB stores only the path.	F-005
Email/SMS gateway	External service for OTP and status notifications.	F-001, F-014

3.5 Chosen Architecture Pattern and Rationale

SCMS uses a layered, modular monolith (Presentation → Application/Service → Data) implemented with Django's Model-Template-View structure and a REST API for the front end. Each SRS module is a separate Django app with its own models, services and tests, so module boundaries are clear while deployment stays a single unit. Microservices were rejected because the expected load (1,000 complaints/day), team size and two-week delivery window do not justify the operational complexity.

Architecture Decision Records

ADR	Decision	Rationale	Alternatives rejected
ADR-001	Layered modular monolith	Simple to build, test and deploy in 2 sprints; clear separation of concerns.	Microservices (over-engineered)
ADR-002	Python Django + Django REST Framework	Built-in auth, ORM, admin, CSRF and password hashing speed up delivery and strengthen security.	Flask (more manual wiring), Laravel
ADR-003	PostgreSQL	Strong integrity constraints, indexing and JSON support for audit details; open source.	MySQL (acceptable alternative)
ADR-004	Server-side sessions with Secure/HttpOnly/SameSite cookies, 15-min idle timeout	Directly satisfies SR-004; revocable on server.	JWT in local storage (XSS risk)
ADR-005	Notification via adapter + background task; console/mock gateway in dev	Complaint lodging must not fail or slow down if the email/SMS provider is down.	Synchronous sending
ADR-006	Attachments on file storage, metadata in DB, validated by type and size	Keeps DB small; prevents malicious uploads.	BLOBs in database

3.6 Technology Stack & Data Stores

Layer	Technology
Front end	HTML5, CSS3 (responsive, WCAG 2.1 AA), JavaScript, Django templates / Bootstrap
Back end	Python 3.11+, Django 5, Django REST Framework, Gunicorn
Database	PostgreSQL 15 (SQLite acceptable for local development)
File storage	Local protected media directory (S3-compatible storage for production)
Security	HTTPS/TLS 1.2+ via Nginx, Argon2/bcrypt hashing, Django CSRF and session framework
Notifications	SMTP email; SMS via provider API (Twilio/Fast2SMS) or mock in development

Layer	Technology
Quality / DevOps	GitHub (repo, Projects, Actions), pytest, flake8, Locust/k6 (performance), axe/Lighthouse (accessibility)

3.7 Risks & Mitigations

Risk	Level	Mitigation
Email/SMS provider unavailable or rate-limited	Medium	Async queue with retry; record FAILED status; show in-app status regardless.
Schedule too short for all 25 requirements in 2 sprints	High	Prioritise High items in Sprint-1; Medium items in Sprint-2; freeze scope on 19 Oct.
Malicious or oversized file uploads	Medium	Whitelist JPG/PNG/PDF, size limit, rename files, store outside web root.
Unauthorised access across roles	High	RBAC enforced server-side on every endpoint (SR-003); negative tests TC-Sec-03.
Performance degradation at 1,000 complaints/day	Medium	DB indexes on tracking_id, status, department, date; pagination; load test TC-Perf-01/02.
Data loss or downtime	Medium	Daily DB backups, health checks, restart policy; availability target 99.5%.
Merge conflicts between team members	Low	Feature branches, small PRs, owner review, GitHub Actions checks.

3.8 Traceability to Requirements

Req ID	Requirement	Design ID	Component	Test Case
SCMS-F-001	Register account	DS-Auth-01	AuthModule	TC-Auth-01
SCMS-F-002	User login	DS-Auth-02	AuthModule	TC-Auth-02
SCMS-F-003	Role-based access	DS-Auth-03	AuthModule	TC-Auth-03
SCMS-F-004	Lodge complaint	DS-Complaint-01	ComplaintModule	TC-Lodge-01
SCMS-F-005	Attach evidence	DS-Complaint-02	ComplaintModule + File storage	TC-Lodge-02
SCMS-F-006	Generate tracking ID	DS-Complaint-03	ComplaintModule	TC-Lodge-03
SCMS-F-007	Track complaint	DS-Track-01	TrackingModule	TC-Track-01
SCMS-F-008	Status history	DS-Track-02	TrackingModule	TC-Track-02
SCMS-F-009	Search/filter	DS-Track-03	TrackingModule	TC-Track-03
SCMS-F-010	Auto assignment	DS-Route-01	RoutingModule	TC-Assign-01
SCMS-F-011	Reassignment	DS-Route-02	RoutingModule	TC-Assign-02
SCMS-F-012	Update status	DS-Admin-01	AdminModule	TC-Admin-01
SCMS-F-013	Close complaint	DS-Admin-02	AdminModule	TC-Admin-02
SCMS-F-014	Status notification	DS-Notify-01	NotificationModule	TC-Notify-01
SCMS-F-015	Reports / feedback	DS-Report-01	Reporting / Feedback	TC-Report-01 / TC-Feedback-01
SCMS-NF-001	Response time	DS-Perf-01	Web UI / API, DB indexes	TC-Perf-01
SCMS-NF-002	Availability	DS-Reliability-01	Deployment, health checks	TC-Ops-01

Req ID	Requirement	Design ID	Component	Test Case
SCMS-NF-003	Scalability	DS-Perf-02	Web UI / API, DB	TC-Perf-02
SCMS-NF-004	Accessibility	DS-UX-01	Web UI	TC-UX-01
SCMS-NF-005	Data retention	DS-Data-01	Database / Archive	TC-Ops-02
SCMS-SR-001	TLS	DS-Sec-01	SecurityModule	TC-Sec-01
SCMS-SR-002	Password hashing	DS-Sec-02	AuthModule	TC-Sec-02
SCMS-SR-003	RBAC	DS-Sec-03	AuthModule + SecurityModule	TC-Sec-03
SCMS-SR-004	Session security	DS-Sec-04	AuthModule	TC-Sec-04
SCMS-SR-005	Audit log	DS-Sec-05	AuditModule	TC-Sec-05

3.9 Security Architecture

Threat modelling using STRIDE:

Threat	SCMS scenario	Mitigation
Spoofing	Attacker logs in as a citizen/staff or registers with a fake contact	OTP verification at registration; salted Argon2/bcrypt hashes; login throttling; session cookies Secure+HttpOnly+SameSite (SR-002, SR-004)
Tampering	Altering complaint status/history or request parameters	Server-side validation; workflow-enforced transitions; append-only status history; CSRF tokens; HTTPS (SR-001)
Repudiation	Staff/admin denies changing or closing a complaint	Audit log with timestamp, user ID and action details (SR-005); history stores actor
Information disclosure	Citizen sees another's complaint; PII or secrets in logs	TLS 1.2+; object-level ownership check; no passwords/OTPs in logs; attachments served only through authorised view
Denial of service	Flooding lodge/login/OTP endpoints; huge uploads	Rate limiting; upload size limit; pagination; request timeouts
Elevation of privilege	Citizen calls staff/admin APIs	RBAC enforced on server for every protected endpoint (SR-003); deny by default; negative test cases

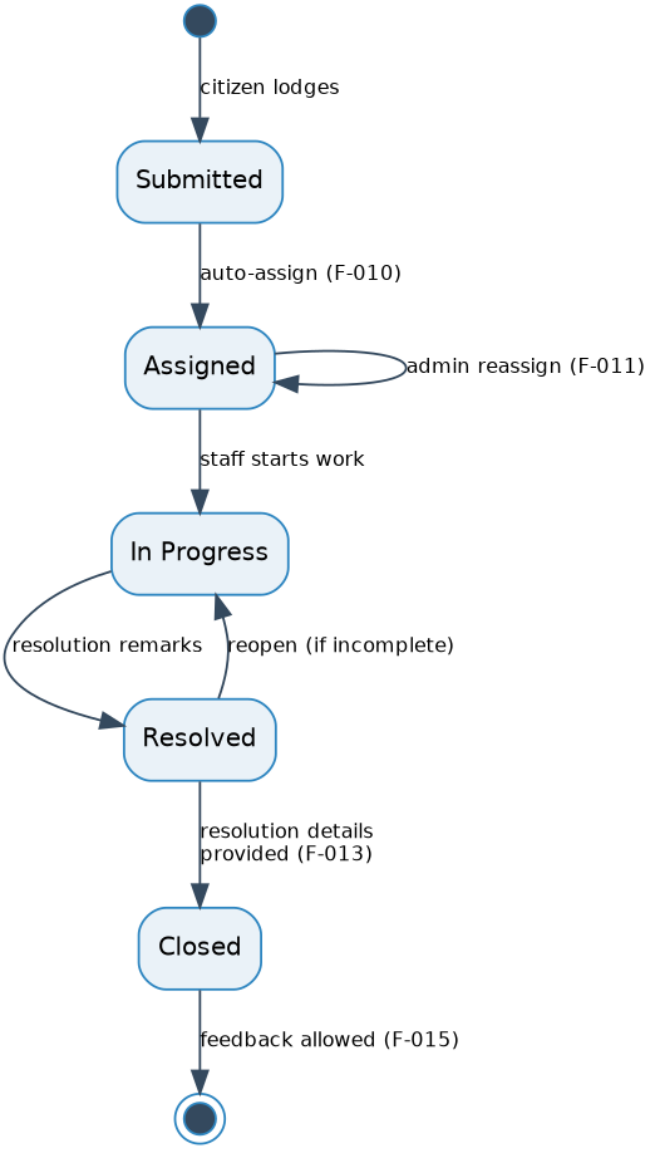
Additional controls: parameterised queries via ORM (SQL injection), output encoding and CSP (XSS), secrets in environment variables, DB least-privilege account, dependency scanning in GitHub Actions.

4. Design

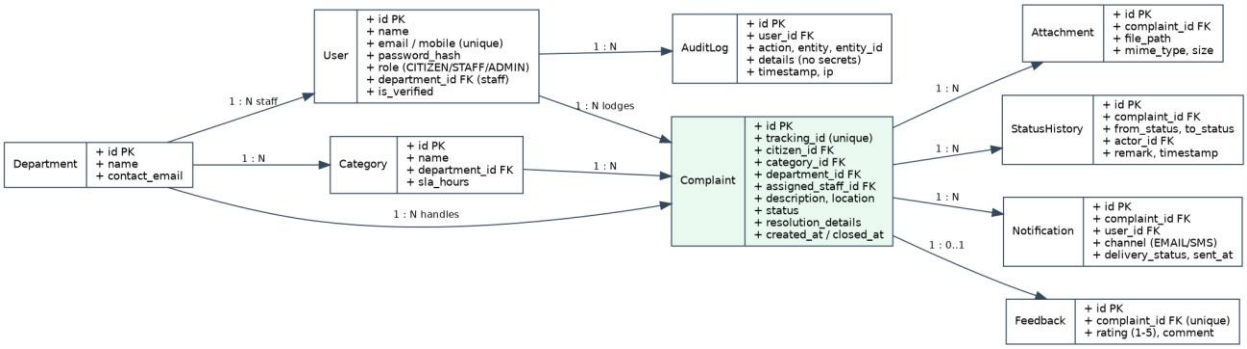
4.1 Design Overview

SCMS is designed as seven functional Django apps (auth, complaints, tracking, routing, admin ops, notifications, reporting) plus shared security/audit utilities. Each app follows the same internal structure: models (data), services (business rules), serializers/views (API and pages) and tests. Business rules such as workflow transitions, tracking ID generation and routing live in the service layer so they can be unit tested without HTTP.

Complaint workflow (state diagram)



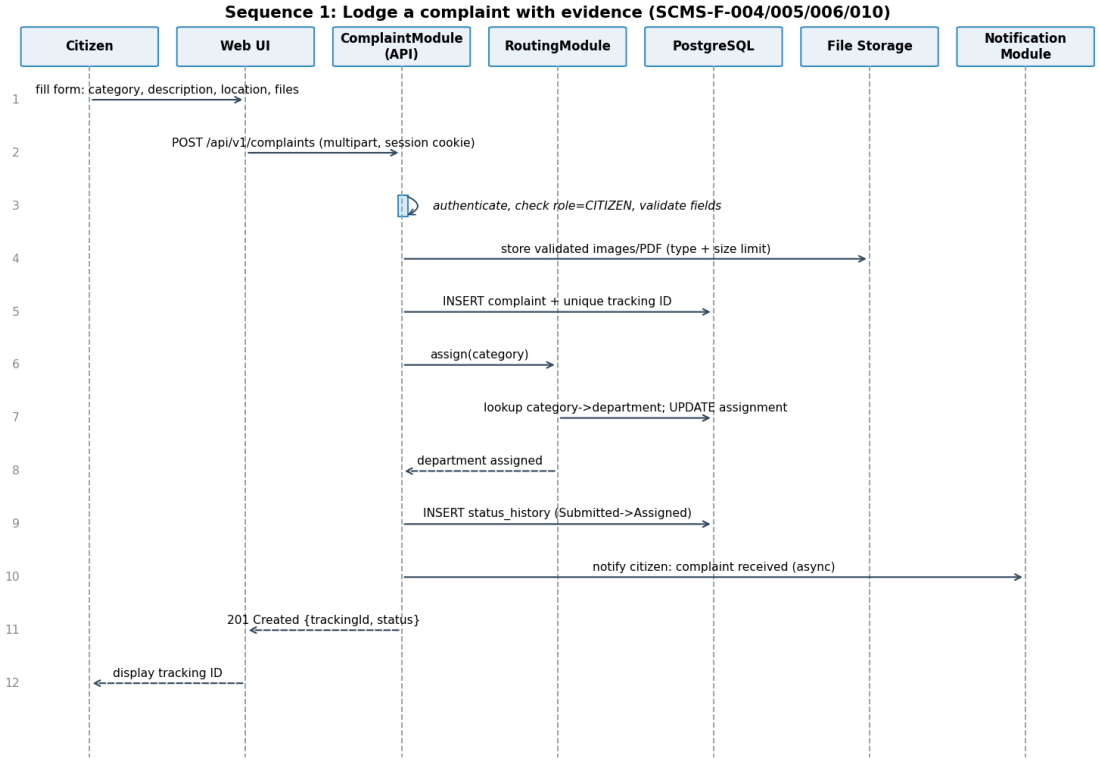
Data model

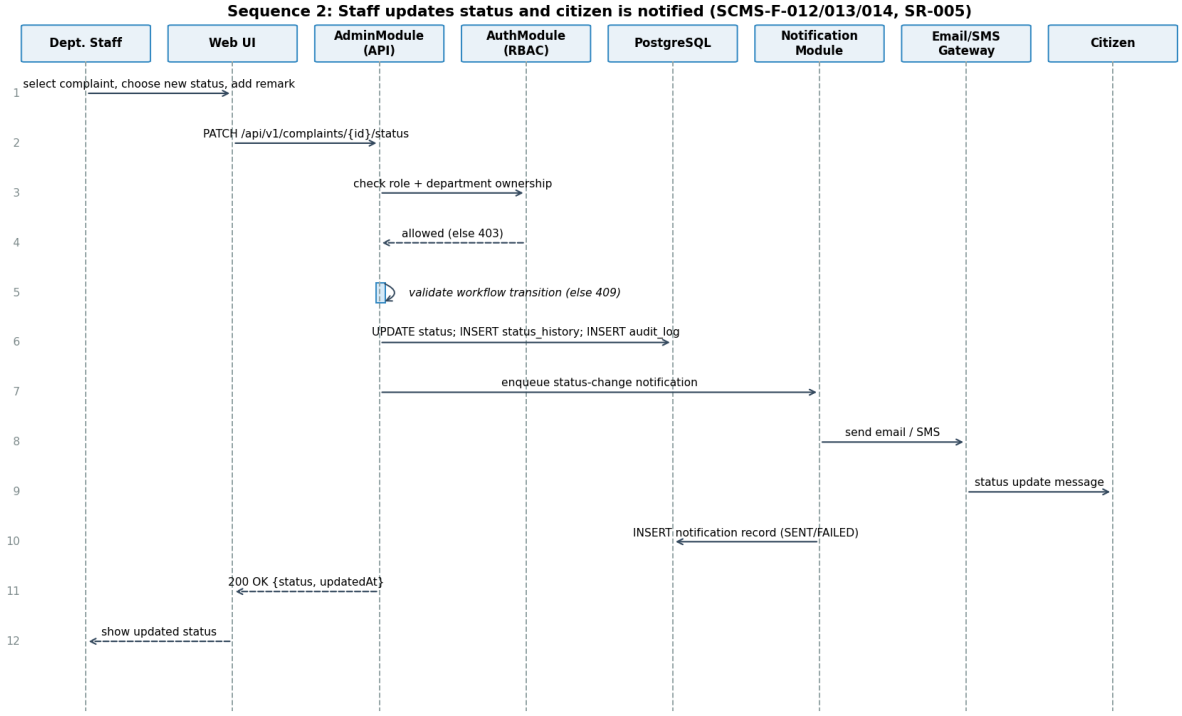


Tracking ID format: SCMS-YYYYMMDD-XXXXXX (6 random alphanumeric characters) with a unique database constraint; regenerated on collision. Complaint, StatusHistory and AuditLog rows are never deleted (5-year retention, NF-005).

4.2 UML Sequence Diagrams

Two project-specific flows are shown: lodging a complaint (citizen flow) and updating status with notification (staff flow).





4.3 API Design

REST/JSON over HTTPS, base path /api/v1. Authentication via session cookie; all state-changing calls require a CSRF token. Interfaces are defined for the Auth, Complaint, Tracking, Admin and Reporting components.

Method	Endpoint	Role	Request	Success / Errors
POST	/auth/register	Public	{name, email|mobile, password}	201 {userId, otpSent} / 400, 409 duplicate
POST	/auth/verify-otp	Public	{userId, otp}	200 verified / 400 invalid or expired
POST	/auth/login	Public	{identifier, password}	200 {role} + session cookie / 401, 429
POST	/auth/logout	Any	–	204 / 401
POST	/complaints	Citizen	multipart {categoryId, description, location, files[]}	201 {trackingId, status, department} / 400, 401, 403, 413, 415
GET	/complaints/track/{trackingId}	Owner / Staff / Admin	–	200 {status, history[], department} / 401, 403, 404
GET	/complaints?id=&category=&status=&from=&to=	Staff / Admin	query filters, page	200 {items[], page} / 401, 403
PATCH	/complaints/{id}/status	Staff / Admin	{status, remark}	200 {status, updatedAt} / 403, 404, 409 invalid transition
POST	/complaints/{id}/reassign	Admin	{departmentId, staffId?}	200 {assignedTo} / 403, 404, 422
POST	/complaints/{id}/close	Staff / Admin	{resolutionDetails}	200 {status: Closed} / 403, 409, 422 missing details
POST	/complaints/{id}/feedback	Citizen (owner)	{rating 1-5, comment}	201 / 403, 409 not closed or already submitted
GET	/reports/complaints?from=&to=&groupBy=	Admin	query	200 {rows[]} / 403
GET	/audit-logs	Admin	query filters	200 {entries[]} / 403

Example 1 – Lodge complaint (ComplaintModule)

```
POST /api/v1/complaints   Content-Type: multipart/form-data
categoryId=3, description="Streetlight not working", location="Sector 12, Main Road",
files[]=photo.jpg

201 Created
{ "trackingId": "SCMS-20261019-K7P2QX", "status": "Assigned", "department": "Electricity" }

Errors: 400 missing mandatory field | 401 not logged in | 403 not a citizen | 413 file too
large | 415 unsupported file type
```

Example 2 – Update status (AdminModule)

```
PATCH /api/v1/complaints/41/status
{ "status": "In Progress", "remark": "Team dispatched" }

200 OK
{ "status": "In Progress", "updatedAt": "2026-10-20T10:42:00Z" }

Errors: 403 not assigned department | 404 unknown complaint | 409 invalid workflow transition
```

4.4 Error Handling, Logging & Monitoring

- Standard error body: { "error": { "code": "INVALID_TRANSITION", "message": "Cannot move from Submitted to Closed", "requestId": "..." } }. Messages are user-friendly and never reveal stack traces, SQL or whether an account exists.
- Field-level validation errors return 400/422 with a list of fields; web forms display inline messages.
- Logging: structured application logs with request ID; passwords, OTPs, session IDs and full contact details are never logged. Business-critical actions go to the AuditLog table (SR-005).
- Notification failures are recorded and retried (max 3 attempts) without failing the main request.
- Monitoring: health endpoint /health, uptime (target 99.5%), p95 response time (target 3 s), error rate, notification failure rate, complaints per day, SLA breaches per department.

4.5 UX Design

- Role-specific views: Citizen (register/login, lodge form, track by ID, history timeline, feedback); Staff (assigned complaints queue, filters, update/close form); Admin (dashboard, users, categories/SLA, reassignment, reports, audit log).
- Simple forms with clear labels, inline validation, a confirmation page showing the tracking ID, and a status timeline for history.
- Responsive layout (mobile-first) that works on desktop and phone browsers.
- Accessibility (WCAG 2.1 AA): semantic HTML, keyboard navigation, visible focus, colour contrast ≥ 4.5:1, alt text and labels, text not conveyed by colour alone; verified with Lighthouse/axe (TC-UX-01).
- Session timeout warning shown before the 15-minute idle logout.

4.6 Open Issues & Next Steps

- Decide the SMS provider (or keep mock) based on cost and availability.
- Confirm the exact category-to-department mapping and SLA hours with the instructor/stakeholders.
- Future enhancements: SLA breach escalation, multilingual UI, GPS/map location picker, mobile app, analytics charts.
- Any deviation found during Sprint-1/Sprint-2 will be logged and explained in the final demo (see Part-2 deliverables).

5. Appendices

5.1 Glossary: SCMS, SLA, RBAC, TLS, OTP, ADR, STRIDE, WCAG, REST, ORM (see Section 1.4).

5.2 References: SCMS SRS v1.0; IEEE 42010; OWASP ASVS and Top 10; NIST SP 800-160; WCAG 2.1; Django documentation.

5.3 Tools: Graphviz/PlantUML, draw.io, Swagger/OpenAPI, GitHub Projects and Actions, pytest.